

# MySQL Assignment–5: Easy Complete Notes

These notes are created directly from the SQL work in your uploaded Assignment-5\_SQL.docx, including the same database, table structure, employee data, queries, outputs, and mistakes shown in the file.

## 1. MySQL Shell Basics

### SQL mode

```
\sql
```

### Connect to MySQL

```
\connect root@localhost:3306
```

### Important

In your file, adding a semicolon after the connection URI caused an invalid-URI error. The working command shown is without the semicolon.

### Check databases

```
SHOW DATABASES;
```

### Select database

```
USE company_db;
```

## 2. Create Database

### Command

```
CREATE DATABASE IF NOT EXISTS company_db;  
USE company_db;
```

### Meaning

IF NOT EXISTS prevents an error if the database already exists.

## 3. Create employees Table

### Command

```
CREATE TABLE employees (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(50) NOT NULL,  
    emp_age INT,  
    emp_department VARCHAR(50),  
    emp_salary DECIMAL(10, 2),  
    emp_city VARCHAR(50)  
);
```

### Column meaning

emp\_id = unique employee ID; emp\_name = employee name; emp\_age = age; emp\_department = department; emp\_salary = salary; emp\_city = city.

### Check table

```
SHOW TABLES;
```

## 4. Insert 10 Employees

### Command

```
INSERT INTO employees  
(emp_id, emp_name, emp_age, emp_department, emp_salary, emp_city)  
VALUES  
(101, 'Arun', 24, 'IT', 65000.00, 'Lucknow'),  
(102, 'K.M.', 24, 'HR', 45000.00, 'Maharajganj'),  
(103, 'Anshuman', 23, 'SED', 55000.00, 'kushinagar'),
```

```
(104, 'Anil', 25, 'WebDev', 35000.00, 'Mirjapur'),  
(105, 'Vikas', 29, 'IT', 48000.00, 'Delhi'),  
(106, 'Neha', 23, 'HR', 38000.00, 'Lucknow'),  
(107, 'Suresh', 35, 'Finance', 80000.00, 'Lucknow'),  
(108, 'Alok', 27, 'IT', 55000.00, 'Kanpur'),  
(109, 'Priya', 30, 'Marketing', 42000.00, 'Delhi'),  
(110, 'Rohan', 25, 'IT', NULL, 'Lucknow');
```

## Result

10 rows were inserted successfully in the uploaded work.

## 5. Display Data

### Correct command

```
SELECT * FROM employees;
```

### Common mistake from the file

```
SELECT * FROM employee; gives an error because the table is named employees (plural), not employee.
```

## 6. UPDATE – Change Salary

### Command

```
UPDATE employees  
SET emp_salary = 50000.00  
WHERE emp_id = 110;
```

### Meaning

Only employee 110 (Rohan) gets the salary changed from NULL to 50000.

## 7. UPDATE – Change City

### Command

```
UPDATE employees  
SET emp_city = 'Delhi'  
WHERE emp_id = 108;
```

### Meaning

Employee 108 (Alok) is moved to Delhi.

## 8. UPDATE – Change Multiple Columns

### Correct syntax

```
UPDATE employees  
SET emp_name = 'Aman', emp_city = 'Maharajganj'  
WHERE emp_id = 107;
```

### Important mistake

Do not write SET twice. The uploaded file shows errors caused by using SET emp\_name=... SET emp\_city=... and SET emp\_name=..., SET emp\_city=.... Multiple columns are separated with commas inside one SET clause.

## 9. DELETE

### Command

```
DELETE FROM employees  
WHERE emp_id = 109;
```

### Meaning

Employee 109 (Priya) is deleted. After this, the table contains 9 employees.

## 10. WHERE – Filtering

### Salary > 50,000

```
SELECT * FROM employees WHERE emp_salary > 50000;
```

### Department = IT

```
SELECT * FROM employees WHERE emp_department = 'IT';
```

### City = Lucknow

```
SELECT * FROM employees WHERE emp_city = 'Lucknow';
```

### Salary between 40,000 and 60,000

```
SELECT * FROM employees WHERE emp_salary BETWEEN 40000 AND 60000;
```

### Name starts with A

```
SELECT * FROM employees WHERE emp_name LIKE 'A%';
```

### Name ends with a

```
SELECT * FROM employees WHERE emp_name LIKE '%a';
```

### Department IT or HR

```
SELECT * FROM employees WHERE emp_department IN ('IT', 'HR');
```

### Age >25 AND salary >50,000

```
SELECT * FROM employees  
WHERE emp_age > 25 AND emp_salary > 50000;
```

### Salary is NULL

```
SELECT * FROM employees WHERE emp_salary IS NULL;
```

### Salary is NOT NULL

```
SELECT * FROM employees WHERE emp_salary IS NOT NULL;
```

### Easy rule

Use = for a normal value, LIKE for patterns, IN for a list of values, BETWEEN for a range, and IS NULL / IS NOT NULL for NULL checking.

## 11. ORDER BY – Sorting

### Lowest to highest salary

```
SELECT * FROM employees ORDER BY emp_salary ASC;
```

### Highest to lowest salary

```
SELECT * FROM employees ORDER BY emp_salary DESC;
```

### Meaning

ASC = ascending; DESC = descending.

## 12. GROUP BY – Department

### Count employees in each department

```
SELECT emp_department, COUNT(*) AS total_employees  
FROM employees  
GROUP BY emp_department;
```

### Average salary per department

```
SELECT emp_department, AVG(emp_salary) AS average_salary  
FROM employees  
GROUP BY emp_department;
```

## Maximum salary per department

```
SELECT emp_department, MAX(emp_salary) AS max_salary
FROM employees
GROUP BY emp_department;
```

## More than 5 employees

```
SELECT emp_department, COUNT(*) AS employee_count
FROM employees
GROUP BY emp_department
HAVING COUNT(*) > 5;
```

## Average salary >60,000

```
SELECT emp_department, AVG(emp_salary) AS avg_salary
FROM employees
GROUP BY emp_department
HAVING AVG(emp_salary) > 60000;
```

## Sort by average salary

```
SELECT emp_department, AVG(emp_salary) AS avg_salary
FROM employees
GROUP BY emp_department
ORDER BY avg_salary DESC;
```

## Key difference

WHERE filters rows before grouping. HAVING filters grouped results after GROUP BY.

## 13. GROUP BY – City

### Number of employees in every city

```
SELECT emp_city, COUNT(*) AS employee_count
FROM employees
GROUP BY emp_city;
```

### Cities with more than 3 employees

```
SELECT emp_city, COUNT(*) AS employee_count
FROM employees
GROUP BY emp_city
HAVING COUNT(*) > 3;
```

## 14. Aggregate Functions

### Total employees

```
SELECT COUNT(*) AS total_employees FROM employees;
```

### Total salary

```
SELECT SUM(emp_salary) AS total_salary_expense FROM employees;
```

### Average salary

```
SELECT AVG(emp_salary) AS overall_average_salary FROM employees;
```

### Highest salary

```
SELECT MAX(emp_salary) AS highest_salary FROM employees;
```

### Lowest salary

```
SELECT MIN(emp_salary) AS lowest_salary FROM employees;
```

### Average IT salary

```
SELECT AVG(emp_salary) AS it_average_salary
FROM employees
WHERE emp_department = 'IT';
```

### Highest HR salary

```
SELECT MAX(emp_salary) AS hr_highest_salary
```

```
FROM employees
WHERE emp_department = 'HR';
```

### Finance total salary

```
SELECT SUM(emp_salary) AS finance_total_salary
FROM employees
WHERE emp_department = 'Finance';
```

### Employees in Delhi

```
SELECT COUNT(*) AS delhi_employee_count
FROM employees
WHERE emp_city = 'Delhi';
```

### Average salary >50,000

```
SELECT AVG(emp_salary) AS avg_salary_above_50k
FROM employees
WHERE emp_salary > 50000;
```

## 15. Department-wise Salary

### Total salary by department

```
SELECT emp_department, SUM(emp_salary) AS total_department_salary
FROM employees
GROUP BY emp_department;
```

### Average salary by department

```
SELECT emp_department, AVG(emp_salary) AS avg_department_salary
FROM employees
GROUP BY emp_department;
```

### Department with highest average salary

```
SELECT emp_department, AVG(emp_salary) AS highest_avg_salary
FROM employees
GROUP BY emp_department
ORDER BY highest_avg_salary DESC
LIMIT 1;
```

### Department with highest total salary

```
SELECT emp_department, SUM(emp_salary) AS highest_total_salary
FROM employees
GROUP BY emp_department
ORDER BY highest_total_salary DESC
LIMIT 1;
```

### City-wise employee count

```
SELECT emp_city, COUNT(*) AS total_employees
FROM employees
GROUP BY emp_city;
```

## 16. Results from Your Uploaded Data

### Department count

IT = 4, HR = 2, SED = 1, WebDev = 1, Finance = 1.

### Department average salary

Finance = 80000; SED = 55000; IT = 54500; HR = 41500; WebDev = 35000.

### Department total salary

IT = 218000; HR = 83000; SED = 55000; WebDev = 35000; Finance = 80000.

### Highest average salary

Finance = 80000.

## Highest total salary

IT = 218000.

## City count

Lucknow = 3, Maharajganj = 2, kushinagar = 1, Mirjapur = 1, Delhi = 2.

## Overall results

Total employees = 9; total salary = 471000; average salary = 52333.333333; maximum salary = 80000; minimum salary = 35000; IT average = 54500; HR maximum = 45000; Finance total = 80000; Delhi employees = 2; average salary above 50000 = 63750.

# 17. Quick Revision Cheat Sheet

## DDL

CREATE DATABASE, CREATE TABLE

## DML

INSERT, UPDATE, DELETE

## DQL

SELECT

## Filtering

WHERE, LIKE, IN, BETWEEN, IS NULL

## Sorting

ORDER BY ASC / DESC

## Grouping

GROUP BY

## Group filtering

HAVING

## Functions

COUNT, SUM, AVG, MAX, MIN

## Top result

ORDER BY ... DESC LIMIT 1

## Practice Order

1. Connect to MySQL.
2. SHOW DATABASES;
3. Create/use company\_db.
4. Create employees.
5. Insert 10 employees.
6. SELECT \* FROM employees;
7. Practice UPDATE and DELETE.
8. Practice WHERE conditions.
9. Practice ORDER BY.
10. Practice GROUP BY + HAVING.
11. Practice COUNT/SUM/AVG/MAX/MIN.
12. Verify your final results.